

Benchmark di Tecnologie Big Data

Progetto 2 – Corso di Big Data (A.A. 2025/2026)

Alessandro Cavina

Matricola: 578902

Repository GitHub: <https://github.com/Alessandro2801/Flights-BigData-Benchmark>

1 Introduzione e Obiettivi del Progetto

Il presente elaborato documenta la progettazione, lo sviluppo e l'analisi sperimentale di una pipeline automatizzata di benchmark orientata all'elaborazione massiva di dati aerei commerciali. L'obiettivo accademico e ingegneristico dello studio consiste nel mettere a confronto, in termini di espressività sintattica, semplicità implementativa, efficienza computazionale e scalabilità, tre dei principali paradigmi di calcolo distribuiti appartenenti all'ecosistema Apache:

- **Apache Spark Core (RDD):** Paradigma di basso livello basato su strutture dati resilienti e distribuite (*Resilient Distributed Datasets*), caratterizzato da un controllo granulare sulle trasformazioni e su logiche di manipolazione di tipo imperativo.
- **Apache Spark SQL (DataFrame):** Framework dichiarativo di alto livello orientato all'ottimizzazione automatica dei piani di esecuzione e dei grafi logici tramite il motore relazionale *Catalyst Optimization Engine*.
- **Apache Hive:** Infrastruttura di Data Warehousing relazionale distribuita. Al fine di massimizzare le performance ed evitare l'obsoleto overhead del framework MapReduce classico, le query in linguaggio *HiveQL* (HQL) sono state eseguite sfruttando il motore di esecuzione di nuova generazione **Apache Tez**, basato sulla modellazione dei job tramite grafi diretti aciclici (DAG).

L'analisi comparativa è stata condotta sottoponendo i tre motori di calcolo a carichi di lavoro identici all'interno di due contesti infrastrutturali distinti, evidenziando il comportamento delle diverse tecnologie al variare del contesto esecutivo:

1. **Ambiente Locale Pseudo-Distribuito:** Configurazione dell'intero stack software (Hadoop 3.4.1, Spark 3.5.5, Hive 4.0.0) operante su una singola macchina host fisica con sistema operativo Ubuntu. In questo scenario, la persistenza poggia sul file system distribuito *HDFS*, mentre l'orchestrazione delle risorse può essere isolata tramite computazione multi-thread nativa (`local[*]`) o sottomessa formalmente al resource manager locale *Hadoop YARN*.
2. **Ambiente Cloud Distribuito (AWS EMR):** Deployment dell'architettura all'interno di un cluster elastico reale gestito tramite il servizio *Amazon Elastic MapReduce* (EMR). L'infrastruttura hardware prevede un'architettura a tre nodi basata su istanze EC2 di tipo `m5.xlarge` (ciascuna dotata di 4 vCPU, 16 GB di memoria RAM e storage EBS dedicato), ripartite in 1 *Primary Node* (Master) e 2 *Core Nodes* (Worker).

1.1 Analisi del Dataset Utilizzato

In conformità con gli obiettivi di analisi sperimentale su dati reali, il benchmark si basa sul popolare dataset pubblico "*Flight Delay Dataset — 2024*" disponibile sulla piattaforma Kaggle al seguente indirizzo: <https://www.kaggle.com/datasets/hrishitpatil/flight-data-2024>.

Il file raccoglie i record dettagliati di tutti i voli di linea interni degli Stati Uniti, tracciando ritardi, cancellazioni e metriche operative delle compagnie aeree.

L'estensione originaria del dataset conta oltre 7 milioni di record e 35 colonne complessive in formato CSV. Al fine di condurre uno studio rigoroso sulla scalabilità verticale e orizzontale come espressamente suggerito dalle indicazioni sperimentali, la matrice dei dati è stata campionata in 5 frazioni progressive a dimensionalità controllata e crescente:

- `flights_1.csv`: porzione campionata pari all'1% dei record complessivi;
- `flights_20.csv`: porzione campionata pari al 20% dei record complessivi;
- `flights_50.csv`: porzione campionata pari al 50% dei record complessivi;
- `flights_70.csv`: porzione campionata pari al 70% dei record complessivi;
- `flights_cleaned.csv`: 100% del volume del dataset originale sottoposto ad un porcesso di preprocessing.

1.2 Definizione dei Job di Analisi

Al fine di testare in modo comparativo le potenzialità computazionali dei diversi motori analitici selezionati, sono state ingegnerizzate due distinte pipeline di elaborazione che riflettono altrettanti scenari d'uso tipici nei sistemi di Data Warehousing e Business Intelligence. Nello specifico, i due job implementati sono così strutturati:

1. **Statistiche delle compagnie aeree (Job 3.1):** Questa elaborazione esegue un'aggregazione dei dati su base granulare per ciascun vettore aereo e per ciascun aeroporto di partenza servito. Il job estrae il codice univoco della compagnia e calcola la lista delle tratte aggregate indicando: il numero totale di voli intercettati, le metriche di ritardo minimo, massimo e medio registrate all'arrivo, il tasso percentuale di cancellazione dei voli e l'elenco cronologico dei mesi in cui il vettore opera su quello specifico scalo.
2. **Report dei ritardi per aeroporto e periodo temporale (Job 3.2):** Questa elaborazione è focalizzata sulla categorizzazione temporale e geografica dei disservizi logistici, strutturando un report mensile per ogni aeroporto di origine. Il job esegue una partizione dei dati per calcolare il numero di voli ripartiti in tre fasce di ritardo in partenza (*Basso*: meno di 15 min, *Medio*: tra 15 e 60 min, *Alto*: oltre 60 min), il rispettivo valore medio del ritardo registrato sia in partenza sia in arrivo per ogni fascia e, infine, la classifica delle tre cause di cancellazione o di ritardo più frequenti riportate.

2 Stack Tecnologico

La progettazione e l'implementazione della pipeline di benchmark hanno richiesto la configurazione di un ecosistema software stratificato. La scelta dei componenti è stata guidata dalla necessità di garantire la piena compatibilità tra i framework di calcolo massivo distribuiti e le librerie analitiche del linguaggio Python, assicurando la perfetta riproducibilità degli esperimenti sia in ambiente locale che in ambiente Cloud.

2.1 Framework dell'Ecosistema Big Data

I componenti Core scelti per l'archiviazione distribuita, l'orchestrazione delle risorse e l'esecuzione dei job computazionali sono:

- **Java OpenJDK 21:** Selezionato come ambiente di runtime standard ed essenziale per garantire l'esecuzione stabile di tutti i demoni nativi di Hadoop e Hive.
- **Apache Hadoop 3.4.1:** Utilizzato sia per la gestione dello storage distribuito basato su blocchi tramite *HDFS*, sia per il monitoraggio e l'allocazione dei container computazionali tramite il resource manager *YARN*.
- **Apache Spark 3.5.5:** Il motore di calcolo in-memory parallelo su cui poggiano le implementazioni dei job, sfruttato sia a basso livello tramite le trasformazioni degli RDD (Spark Core), sia ad alto livello tramite l'ottimizzazione relazionale dei DataFrame (Spark SQL).
- **Apache Hive 4.0.0:** L'infrastruttura di Data Warehousing relazionale adottata per l'esecuzione del codice dichiarativo HiveQL. Al fine di massimizzare l'efficienza, Hive è stato configurato per cooperare con l'engine di nuova generazione **Apache Tez**, basato sull'orchestrazione dei task tramite grafi diretti aciclici (DAG).

2.2 Dipendenze Software Python

Per la gestione dei flussi di ingestion, per la pulizia preliminare dei dati e per la successiva fase di visualizzazione delle metriche di performance, è stato isolato un ambiente virtuale contenente i seguenti moduli installati tramite `pip`:

- `pyspark==3.5.5`: Libreria fondamentale per interfacciare in modo nativo gli script di analisi Python con il core engine di Apache Spark.
- `kagglehub`: Client ufficiale utilizzato per automatizzare le procedure di autenticazione e il download sicuro del dataset grezzo tramite le API di Kaggle.
- `pandas`: Sfruttato esclusivamente nella pipeline iniziale per le operazioni di manipolazione tabellare e campionamento sul file system locale.
- `matplotlib`: Il motore grafico utilizzato dallo script di monitoraggio per generare e salvare i report comparativi e le curve di scalabilità in formato PNG.

3 Preparazione e Pulizia dei Dati (Data Preprocessing)

La fase di preparazione e pulizia dei dati rappresenta uno dei pilastri fondamentali nell'ingegneria dei Big Data. Lavorare su dataset reali di dimensioni significative comporta inevitabilmente il confronto con dati sporchi, incompleti o ridondanti, che rischiano di compromettere sia l'accuratezza delle analisi sia l'efficienza computazionale dei framework distribuiti. In questo progetto, l'attività di preprocessing è stata progettata per ridurre drasticamente l'overhead di I/O e limitare lo scambio di dati inutili sulla rete durante le fasi di shuffle, preparando una base dati ottimizzata e standardizzata per le computazioni successive.

3.1 Decisioni di Progetto e Logica di Pulizia

In conformità con la flessibilità metodologica concessa dalle note finali della consegna, sono state adottate tre specifiche decisioni architetturali per abbattere il rumore di fondo del dataset e velocizzare i calcoli:

1. **Selezione delle colonne rilevanti e riduzione della dimensionalità:** Il dataset originale si presentava fortemente appesantito da un totale di 35 colonne complessive. Molte di queste features includevano informazioni testuali ridondanti o attributi operativi non legati ai nostri scopi analitici. Per alleggerire drasticamente il carico sui dischi di HDFS e sulla memoria RAM dei nodi operai, sono state selezionate esclusivamente 9 colonne chiave dotate di reale potere informativo per i job (`month`, `op_unique_carrier`, `origin`, `dest`, `dep_delay`, `arr_delay`, `cancelled`, `cancellation_code` e la nuova variabile `delay_code`).
2. **Rimozione dei voli devianti (Outliers):** All'interno dei record sono state individuate righe corrispondenti a voli devianti su altri scali aeroportuali. Queste righe presentavano valori inconsistenti rispetto alle metriche standard di ritardo all'arrivo e alla partenza, configurandosi come veri e propri anomalie statistici (outliers). Al fine di non sporcare le medie complessive e non compromettere la coerenza dei risultati richiesti, tutti i voli devianti sono stati intercettati e rimossi dalla pipeline di campionamento.
3. **Ingegnerizzazione della features `delay_code`:** Il file originario frammentava le motivazioni del ritardo all'interno di 5 colonne distinte, dedicate ciascuna a una specifica causa aziendale o meteorologica. Questa granularità avrebbe costretto i motori di calcolo (specialmente Hive e Spark Core) a eseguire costose computazioni condizionali su più colonne per ogni singola riga. Si è scelto di accorpare ed aggregare queste 5 colonne in un'unica feature sintetica denominata `delay_code`, la quale mappa la causa prevalente del disservizio. Questa normalizzazione ha semplificato drasticamente la scrittura delle query e la logica dei job, riducendo l'ingombro del file e ottimizzando i tempi di esecuzione.

3.2 Generazione delle Frazioni Progressive

Una volta definita la logica di pulizia tramite lo script `preprocessing.py`, è stato eseguito l'algoritmo di campionamento statistico controllato `generate_portions.py` per rispondere direttamente allo studio di scalabilità richiesto dalla consegna. Sfruttando la funzione di partizionamento, il dataset ripulito al 100% (`flights_cleaned.csv`) è stato ridotto in frazioni progressive (1%, 20%, 50% e 70%) stoccate sul file system locale della macchina host.

L'intero processo si conclude con l'automazione dello script `generate_data.sh`, il quale si occupa di creare le directory dedicate all'interno di HDFS e di effettuare il caricamento distribuito dei 5 file CSV pronti per la successiva fase sperimentale.

4 Analisi Sperimentale

L'analisi sperimentale rappresenta la fase cardine del progetto, finalizzata a valutare sul campo le performance e la scalabilità dei tre motori di calcolo analizzati. L'intera suite di test è completamente automatizzata e orchestrata da una pipeline software coordinata dallo script Bash `experiments.sh`, il quale si interfaccia con il core engine Python `benchmark.py`. Questa pipeline si occupa di attivare i job in sequenza su tutte e cinque le frazioni del dataset, registrare in modo millimetrico i tempi di esecuzione (espressi in secondi) e produrre i report grafici finali salvati nella directory `logs/`. L'orchestrazione è stata ingegnerizzata per supportare nativamente tre diverse modalità di sottomissione software (`local[*]`, `yarn` e `aws`), distribuite all'interno di due contesti infrastrutturali hardware.

4.1 Ambiente Locale Pseudo-Distribuito (Host Ubuntu)

Il primo ambiente di test è configurato su un host fisico Ubuntu Linux (16 Core fisici e 22 CPU logiche) che emula un'architettura distribuita. La predisposizione del sistema (pulizia dei residui temporanei, formattazione del NameNode e avvio dei cinque demoni core di HDFS e YARN) è automatizzata tramite lo script `setup.sh`.

In questo contesto si isolano due diverse modalità di calcolo software:

- **Modalità `local[*]` (`bash experiments.sh local[*]`):** Apache Spark opera in multi-threading puro. I task paralleli vengono eseguiti direttamente nella memoria JVM del singolo processo dell'host, sfruttando tutti i core logici della CPU dell'host. Il resource manager YARN viene completamente bypassato.
- **Modalità `yarn` (`bash experiments.sh yarn`):** I job vengono sottomessi formalmente al resource manager locale Hadoop YARN. Questa configurazione simula le restrizioni di un cluster reale, forzando l'allocazione di container di memoria dedicati sulla macchina host ed introducendo l'overhead tipico della gestione distribuita dei task.

4.2 Ambiente Cloud Distribuito (AWS EMR)

Il secondo ambiente sfrutta il servizio gestito **Amazon Elastic MapReduce (EMR)**. Il cluster adotta una topologia personalizzata a tre nodi fisici basati su istanze EC2 di tipo `m5.xlarge` (4 vCPU e 16 GB di RAM ciascuna), con lo stack Hadoop, Spark e Hive preconfigurato sul motore di esecuzione Tez. I ruoli hardware sono così suddivisi:

- **1 Primary Node (Master):** Dedicato esclusivamente al coordinamento logico del cluster e all'hosting dei nodi di controllo di YARN e HDFS.
- **2 Core Nodes (Workers):** Adibiti alla parallelizzazione e all'esecuzione fisica dei task computazionali, con volumi EBS dedicati per lo storage distribuito.

A supporto dell'infrastruttura Cloud, un bucket **Amazon S3** funge da storage di transito. I file CSV processati localmente sono stati caricati su S3 e successivamente trasferiti, via SSH dal Master Node, all'interno dell'HDFS remoto. L'attivazione del comando `bash experiments.sh aws` avvia l'esecuzione nativa per l'ambiente Cloud, storicizzando i risultati e le metriche PNG nel percorso dedicato `logs/aws/`.

5 Risultati

compagnia	aeroporto partenza	numero_voli	ritardo_min_arrivo	ritardo_max_arrivo	ritardo_medio_arrivo	tasso_cancellazione	mesi_operativi
9E	LGA	32011	-69.0	1100.0	-3.05	0.035	1,2,3,4,5,6,7,8,9,10,11,12
9E	ATL	26798	-43.0	1207.0	2.46	0.0109	1,2,3,4,5,6,7,8,9,10,11,12
9E	JFK	17108	-60.0	1082.0	1.1	0.0345	1,2,3,4,5,6,7,8,9,10,11,12
9E	DTW	12712	-46.0	941.0	0.18	0.0093	1,2,3,4,5,6,7,8,9,10,11,12
9E	MSP	7540	-50.0	893.0	3.14	0.0081	1,2,3,4,5,6,7,8,9,10,11,12
9E	CVG	6605	-56.0	1035.0	2.07	0.0174	1,2,3,4,5,6,7,8,9,10,11,12
9E	RDU	6425	-47.0	1018.0	8.06	0.0297	1,2,3,4,5,6,7,8,9,10,11,12
9E	CLT	3133	-53.0	1027.0	9.04	0.0393	1,2,3,4,5,6,7,8,9,10,11,12
9E	TYS	2909	-49.0	781.0	-4.82	0.0172	1,2,3,4,5,6,7,8,9,10,11,12
9E	ROC	2683	-49.0	1184.0	1.37	0.0335	1,2,3,4,5,6,7,8,9,10,11,12

Figura 1: Prime 10 righe di output ottenute per il Job 1.

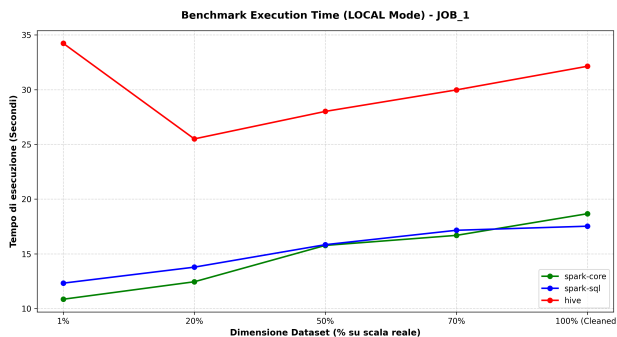
aeroporto	mesi	voli	ritardo_basso	ritardo_medio_dep_basso	ritardo_medio_arr_basso	voli	ritardo_medio	ritardo_medio_dep_medio	ritardo_medio_arr_medio	voli	ritardo_alto	ritardo_medio_dep_alto	ritardo_medio_arr_alto	top 3 cause
ABE	1	275	-5.51	-14.56	30	34.9	32.5	30	241.3	234.03	L,N,C			
ABE	2	296	-6.62	-19.57	19	28.53	10.11	14	227.93	217.93	C,N,L			
ABE	3	339	-6.19	-18.37	28	30.68	19.89	23	173.43	163.39	N,L,C			
ABE	4	303	-6.45	-17.3	29	33.45	27.66	29	258.66	250.38	C,L,N			
ABE	5	320	-6.4	-12.5	26	30.08	29.58	39	183.13	177.13	L,N,C			
ABE	6	335	-6.85	-16.46	37	26.65	17.22	37	173.24	172.32	C,L,N			
ABE	7	280	-6.36	-10.59	41	34.61	30.15	37	138.08	137.08	L,C,N			
ABE	8	289	-6.64	-14.06	42	32.02	27.98	27	146.96	144.3	L,C,N			
ABE	9	276	-6.86	-15.9	24	38.08	32.88	16	265.06	261.25	L,C,W			
ABE	10	302	-6.03	-17.9	31	26.23	11.97	18	193.17	183.83	L,C,B			

only showing top 10 rows

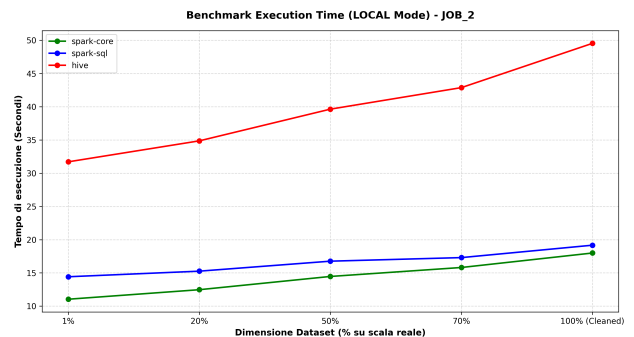
Figura 2: Prime 10 righe di output ottenute per il Job 2.

5.1 Analisi Sperimentale in Modalità Locale Pura (local[*])

In questa fase di test, la pipeline di benchmark è stata eseguita sfruttando la sottomissione in modalità multi-threading nativa sulla macchina host. Questo scenario isola l'esecuzione all'interno della JVM di Spark sfruttando tutti i 22 thread logici resi disponibili dall'architettura ibrida del processore dell'host, escludendo l'overhead di allocazione dei container YARN. Di seguito vengono riportati i grafici comparativi dei tempi di esecuzione rilevati per entrambi i job analitici al variare della dimensionalità dell'input.



(a) Andamento tempi per il Job 1



(b) Andamento tempi per il Job 2

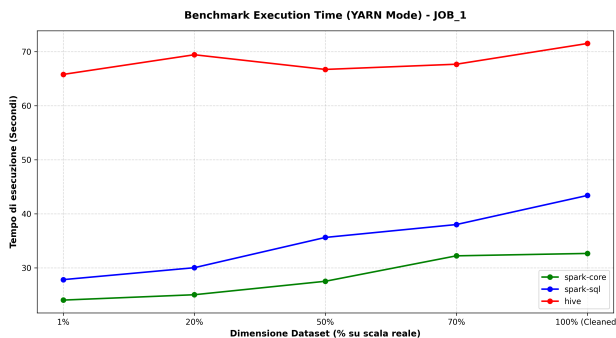
Figura 3: Confronto prestazionale delle tre tecnologie in modalità local al variare del dataset.

Dando un'occhiata ai grafici, salta subito all'occhio come Spark sia nettamente più veloce di Hive in questa modalità locale. Sia Spark Core che Spark SQL si comportano benissimo,

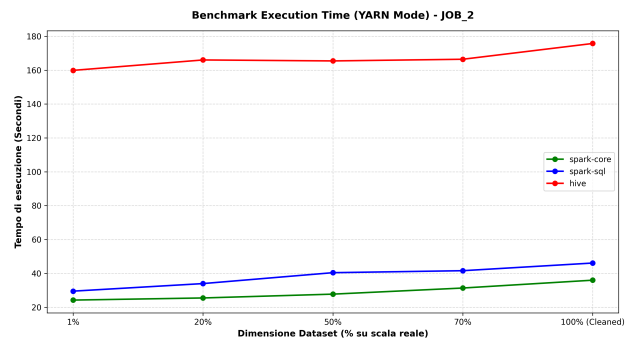
rimanendo sempre molto bassi nei tempi (tra gli 11 e i 19 secondi) e mostrando una crescita davvero minima anche quando il dataset arriva al 100%. In particolare, Spark Core è il più reattivo sui file più piccoli perché non ha layer di astrazione intermedi, mentre Spark SQL si dimostra efficiente sul lungo periodo, arrivando a pareggiare o superare leggermente Core sul dataset completo grazie alle ottimizzazioni automatiche che fa sulle query e sulle aggregazioni. Per quanto riguarda Hive, i tempi sono decisamente più alti e si muovono tra i 25 e i 50 secondi. Nel primo Job si nota una cosa strana: il tempo con l'1% dei dati è il più alto in assoluto (circa 34 secondi) e poi scende con il 20%. Questo succede perché la macchina spreca un sacco di tempo all'inizio solo per avviare l'infrastruttura di Hive e Tez, e questo peso fisso rovina i risultati sui file minuscoli. Nel secondo Job, invece, la curva cresce in modo più classico e lineare perché l'algoritmo è più complesso, confermando che Hive fa un po' di fatica a girare in locale su un singolo host.

5.2 Analisi Sperimentale in Modalità Pseudo-Distribuita (yarn)

In questa seconda sessione di test, la pipeline è stata eseguita sottomettendo i job direttamente al gestore delle risorse Hadoop YARN. A differenza della modalità precedente, questo scenario introduce un carico di lavoro più realistico per un cluster, poiché costringe il sistema ad allocare container di memoria separati sulla macchina per gestire i vari task. Di seguito vengono presentati i due grafici comparativi ottenuti per i rispettivi job computazionali.



(a) Andamento tempi per il Job 1 (YARN)



(b) Andamento tempi per il Job 2 (YARN)

Figura 4: Confronto prestazionale delle tre tecnologie in modalità YARN al variare del dataset.

Guardando i risultati di questa modalità, si nota subito che tutti i tempi di esecuzione si sono alzati rispetto alla modalità locale pura. Questo aumento generale è dovuto principalmente al tempo che YARN impiega per negoziare le risorse, creare i container di calcolo e gestire i vari task, introducendo un overhead fisso che si fa sentire su tutte le tecnologie.

Spark continua a dimostrarsi molto più efficiente e leggero. Sia Spark Core che Spark SQL mantengono un andamento abbastanza stabile e prevedibile su entrambi i job, con tempi che oscillano tra i 24 secondi sui file piccoli e un massimo di circa 45 secondi sul dataset completo. In questo scenario, Spark Core rimane quasi sempre davanti a Spark SQL; questo distacco leggermente più marcato rispetto a prima ci dice che il motore SQL risente un po' di più della combinazione tra l'ottimizzazione della query e la successiva allocazione dei container di YARN. Il vero picco nei tempi lo fa registrare ancora una volta Hive, le cui curve schizzano verso l'alto stabilizzandosi su valori molto pesanti. Nel Job 1 Hive richiede tra i 66 e i 71 secondi stabili, mentre nel Job 2 si parte da ben 160 secondi per toccare quasi i 176 secondi sul dataset al 100%. La cosa interessante è che in modalità YARN le curve di Hive sono quasi piatte. Questo comportamento ci fa capire che il tempo impiegato per elaborare fisicamente i dati è quasi trascurabile rispetto all'enorme overhead fisso richiesto da Hive e Tez per interfacciarsi con YARN e allocare le risorse sul sistema locale.

5.3 Analisi Sperimentale in Ambiente Cloud (aws)

L'ultima sessione di test sposta l'intera pipeline di benchmark su un'infrastruttura Cloud reale utilizzando il servizio gestito Amazon EMR. L'obiettivo di questo scenario è testare i tre motori all'interno di un vero cluster distribuito composto da un Master Node e due Worker Node di tipo `m5.xlarge`, dove i dati sono storicizzati in blocchi nativi HDFS e la potenza di calcolo è parallelizzata su macchine fisiche distinte collegate in rete. Di seguito sono riportati i grafici comparativi dei tempi registrati in ambiente AWS.

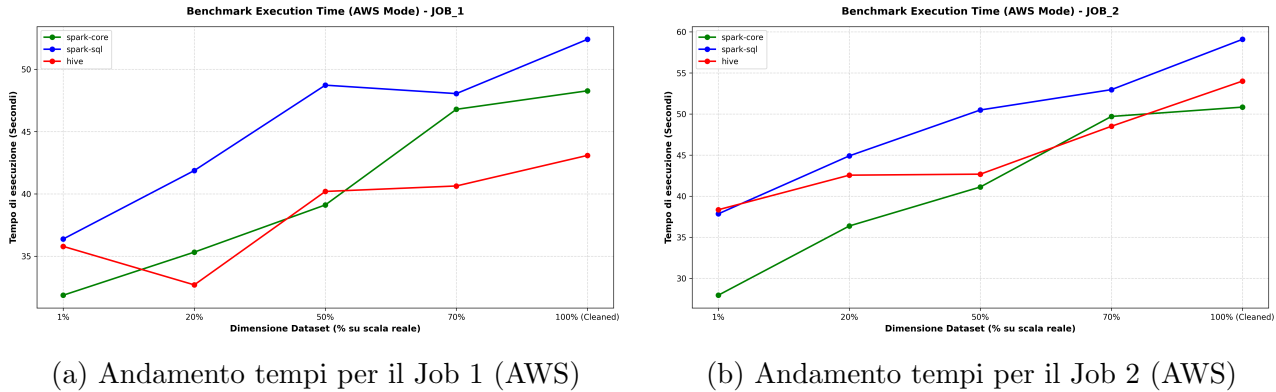


Figura 5: Confronto prestazionale delle tre tecnologie in ambiente AWS EMR al variare del dataset.

I risultati sul Cloud sono molto sorprendenti e mostrano un comportamento totalmente diverso rispetto ai test fatti in locale, specialmente se guardiamo l'incredibile recupero di Apache Hive. In generale, i tempi delle tre tecnologie sono tutti molto vicini e schiacciati in una fascia ristretta, che va da un minimo di 28 secondi a un massimo che non supera mai i 60 secondi, confermando l'ottima capacità di scalabilità del cluster al crescere del dataset.

La vera sorpresa in questo scenario è proprio Hive con l'engine Tez. Se in locale e in modalità YARN era la tecnologia di gran lunga più lenta, qui sul vero cluster distribuito mostra i muscoli: nel Job 1 batte sia Spark Core che Spark SQL a partire dal 50% del dataset, chiudendo al 100% come il motore più veloce in assoluto (circa 43 secondi). Anche nel Job 2 si comporta benissimo, rimanendo praticamente incollato a Spark Core e distaccando nettamente Spark SQL sul dataset completo. Questo riscatto clamoroso dimostra che l'architettura di Hive e i DAG di Tez sono nati proprio per lavorare su cluster veri, dove la frammentazione dei dati su più nodi fisici permette di parallelizzare le letture e le aggregazioni in modo molto più efficiente rispetto a una singola macchina host.

Per quanto riguarda Spark, notiamo un andamento speculare in entrambi i job. Spark Core si conferma eccezionale sui file piccoli (toccando il tempo più basso in assoluto di 28 secondi sull'1% del Job 2), ma tende a salire in modo costante all'aumentare dei dati. Spark SQL, invece, in ambiente Cloud fa registrare le performance meno brillanti del lotto, posizionandosi quasi sempre come la linea più alta nei grafici e toccando il picco di 59 secondi al 100% dell'input. Questo comportamento è causato dal fatto che, su un cluster distribuito reale, l'ottimizzatore Catalyst di Spark SQL genera un piano di esecuzione che risente maggiormente dello scambio di dati sulla rete (fase di Shuffle) tra il nodo Master e i nodi Worker rispetto alla gestione più diretta dei blocchi operata da Hive o da Spark Core.

6 Conclusioni

Questo progetto di benchmark ha permesso di analizzare sul campo, con dati reali, come si comportano diversi motori di calcolo al variare sia della dimensione dell'input che dell'infrastruttura sottostante. I risultati ottenuti nelle tre modalità di esecuzione (`local[*]`, `yarn` e `aws`) offrono diversi spunti di riflessione molto interessanti.

Se guardiamo il confronto tra le tre modalità di calcolo, la modalità locale pura (`local[*]`) si è rivelata in assoluto la più veloce per i framework in-memory come Spark. L'assenza di layer intermedi e la possibilità di far viaggiare i dati direttamente sui thread della CPU dell'host abbatte a zero i tempi morti. Tuttavia, è uno scenario ideale: nel momento in cui siamo passati alla modalità `yarn` sulla stessa macchina, l'introduzione dei container ha fatto schizzare i tempi verso l'alto. Questo ci fa capire quanto pesi l'overhead di gestione, negoziazione e scheduling delle risorse in un ecosistema Hadoop, un fattore che spesso conta più del tempo di calcolo vero e proprio, specialmente sui dataset minori.

Il vero cambio di paradigma si nota però con il passaggio al Cloud su AWS EMR. Qui l'infrastruttura distribuita su tre nodi fisici introduce la latenza della rete, evidenziando il peso delle operazioni di **Shuffle**. Lo scambio di dati tra i nodi durante i raggruppamenti ha infatti penalizzato leggermente Spark SQL, ma ha letteralmente svoltato le performance di Apache Hive. Un ruolo cruciale in tutte le modalità è stato giocato dalla **preparazione dei dati** effettuata a monte: l'eliminazione dei voli devianti (outliers) e la compressione delle colonne delle cause nell'unica feature `delay_code` hanno ridotto drasticamente il volume dei dati scambiati in rete, evitando che le successive operazioni di **aggregazione** mandassero in saturazione la memoria del cluster o della macchina locale.

Nel bilancio tecnologico finale, possiamo riassumere il comportamento dei tre motori in questo modo:

- **Spark Core (RDD)** si è dimostrato il motore più costante e affidabile quando si lavora su una singola macchina o su file medio-piccoli. Controllare i dati a basso livello azzerava i tempi di avvio, anche se richiede un codice più lungo e complesso da scrivere, riducendo la semplicità implementativa.
- **Spark SQL (DataFrame)** offre un'espressività fantastica e una semplicità di scrittura imbattibile. In locale l'ottimizzatore Catalyst fa un lavoro eccellente nelle aggregazioni pesanti pareggiando Spark Core sul dataset al 100%, mentre soffre un po' di più l'ambiente Cloud distribuito a causa dello Shuffle in rete tra i nodi del cluster.
- **Apache Hive (Tez)** si è comportato in modo opposto rispetto a Spark. In ambiente locale e pseudo-distribuito è decisamente penalizzato a causa di tempi di cold-start e di sottomissione dei job mastodontici. Al contrario, nel Cloud su un vero cluster AWS mostra un'eccellente scalabilità, dimostrando che l'accoppiata Hive-Tez esprime il suo massimo potenziale proprio quando può parallelizzare le query relazionali su nodi fisici distinti, arrivando addirittura a battere Spark sul dataset completo.

In conclusione, non esiste una tecnologia migliore in assoluto: Spark Core e SQL rimangono la scelta ideale per elaborazioni rapidi, interattive e in-memory su macchine singole o cluster bilanciati, mentre Hive su Tez si conferma uno strumento potentissimo e robusto per scenari puri di Data Warehousing su cluster distribuiti di grandi dimensioni.